

Exercise 1 (Nonlinear least squares)

Consider the data and the power-law model

$$t = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{bmatrix}, \quad y = \begin{bmatrix} 1.2 \\ 2.8 \\ 5.1 \\ 8.0 \\ 14.7 \end{bmatrix},$$

$$f(t; c) = c_1 t^{c_2}, \quad c = \begin{bmatrix} c_1 \\ c_2 \end{bmatrix}.$$

Part (a)

Define the nonlinear least-squares problem for fitting the model to the data. In particular, write down the residuals and the residual vector $r(c)$.

$$r_i(c) = c_1 t_i^{c_2} - y_i, \quad i = 1, \dots, 5$$

Consider the form below:

$$\begin{bmatrix} t_1 & \cdots & \cdots & 0 \\ 0 & t_2 & & \vdots \\ \vdots & & \ddots & 0 \\ 0 & \cdots & \cdots & t_5 \end{bmatrix}^{c_2} \begin{bmatrix} c_1 \\ c_1 \\ \vdots \\ c_1 \end{bmatrix} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_N \end{bmatrix}$$

$$\implies \vec{r} = \vec{\hat{y}} - \vec{y}$$

Part (b)

Define: $E(c) = \frac{1}{2} \|r(c)\|_2^2$. Set $F(c) = \nabla E(c)$, where $\nabla E(c)$ is regarded as a row vector. Reformulate the stationarity condition as where $F(c)^T$ is a column vector.

$$F(c)^T = 0,$$

Show that where $J(c) = Dr(c)$, and compute the Jacobian matrix $J(c)$ explicitly.

$$F(c)^T = J(c)^T r(c),$$

$$E(c) = \frac{1}{2} \|r(c)\|_2^2 \equiv \frac{1}{2} \vec{r}^T \vec{r} \implies \nabla E := \left(\frac{\partial \vec{r}}{\partial c} \right)^T \vec{r} = J(c)^T \vec{r} = F^T(c)$$

$$\therefore F(c)^T = 0 \iff J(c)r(c) = 0$$

$$c_1 t_i^{c_2} \frac{\partial r_i}{\partial c_1} = t_i^{c_2} \quad c_1 t_i^{c_2} \frac{\partial r_i}{\partial c_2} = c_1 t_i^{c_2} \log(t_i) \implies J := \begin{bmatrix} t_1^{c_2} & c_1 t_1^{c_2} \log(t_1) \\ \vdots & \vdots \\ t_5^{c_2} & c_1 t_5^{c_2} \log(t_5) \end{bmatrix}$$

Part (c/d)

Formulate the Gauss-Newton algorithm for this problem, starting from the initial guess

$$c^{(0)} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

As a hint, if $A = J(c^{(k)})$, then the Gauss-Newton step δc is defined by

$$A^T A \delta c = -A^T r(c^{(k)})$$

$$c^{(k+1)} = c^{(k)} + \delta c.$$

If possible, implement the algorithm in your favorite language (MATLAB, Python, Julia, C/C++, ...). Report the approximate fitted parameters and, if you wish, include a plot of the data together with the fitted curve.

Code/Algorithm cbox

```

1 import numpy as np
2
3 import matplotlib.pyplot as plt
4
5
6 def gauss_newton(residual_func, jacobian_func, c0, tol=1e-6,
   ↪ max_iterations=100):
7     """
8     @Implements the Gauss-Newton method for non-linear least squares.
9     @Parameters:
10         residual_func (callable): Function that returns the residual vector
   ↪ r(c).
11         jacobian_func (callable): Function that returns the Jacobian matrix
   ↪ J(c).
12         c0 (array-like): Initial guess for the parameters c.
13         tol (float): Tolerance for the stopping criterion.
14         max_iterations (int): Maximum number of iterations to perform.
15
16     @Returns: np.ndarray: The optimized parameters c.
17     """
18     # Initialize starting point as a float array

```

```

19     c = np.array(c0, dtype=float)
20
21     for k in range(max_iterations):
22         # Compute the residual vector
23         r = residual_func(c)
24
25         # Compute the Jacobian
26         A = jacobian_func(c)
27
28         # Solve the normal equations for the Gauss-Newton step delta_c:
29         #  $A^T A \delta c = -A^T r$ 
30         A_T = A.T
31         normal_matrix = A_T @ A
32         normal_vector = -A_T @ r
33
34         delta_c = np.linalg.solve(normal_matrix, normal_vector)
35
36         # Update
37         c = c + delta_c
38
39         # Stop when  $\|\delta c\|$  is sufficiently small
40         if np.linalg.norm(delta_c) < tol:
41             print(f"Converged in {k + 1} iterations.")
42             break
43         else:
44             print("Warning: Reached maximum iterations without converging.")
45
46     return c
47
48
49 if __name__ == "__main__":
50     # Fit the model:  $f(t, c) = c_0 * \exp(c_1 * t)$ 
51     # Sample data
52     t_data = np.array([1, 2, 3, 4, 5])
53     y_data = np.array([1.2, 2.8, 5.1, 8.9, 14.7])
54
55     # Define the residual function:  $r(c) = \text{Model}(c) - \text{Data}$ 
56     def calculate_residual(c):
57         return c[0] * np.exp(c[1] * t_data) - y_data
58
59     # Define the Jacobian matrix function
60     def calculate_jacobian(c):
61         # The Jacobian has dimensions (number_of_data_points) x
62         # ↪ (number_of_parameters)
63         J = np.empty((len(t_data), 2))
64         # Partial derivative wrt c0:  $\exp(c_1 * t)$ 

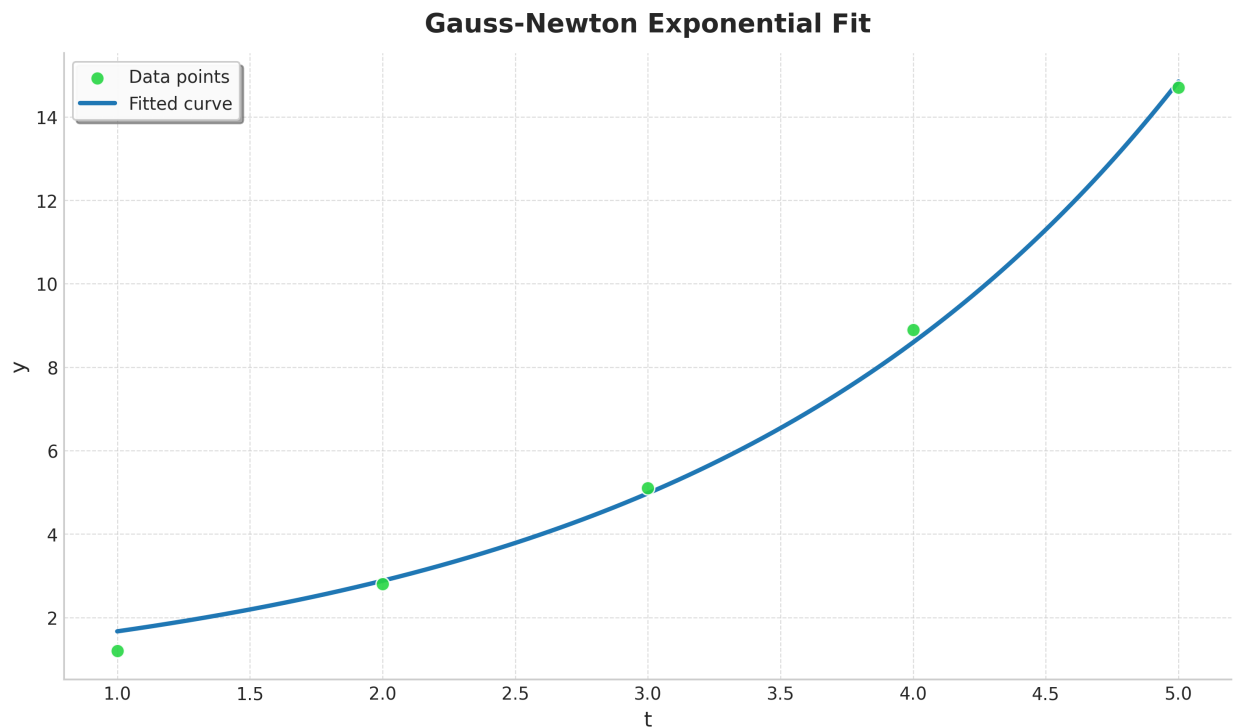
```

```
64     J[:, 0] = np.exp(c[1] * t_data)
65     # Partial derivative wrt c1: c0 * t * exp(c1 * t)
66     J[:, 1] = c[0] * t_data * np.exp(c[1] * t_data)
67
68     return J
69
70 c_initial = [1.0, 1.0]
71
72 c_optimized = gauss_newton(calculate_residual, calculate_jacobian,
73     ↪ c_initial)
74
75 print("\nInitial guess c(0):", c_initial)
76 print("Optimized parameters c:", c_optimized)
77
78 # Plot the data points and fitted curve
79 t_fit = np.linspace(t_data.min(), t_data.max(), 300)
80 y_fit = c_optimized[0] * np.exp(c_optimized[1] * t_fit)
81
82 plt.style.use("seaborn-v0_8-whitegrid")
83 fig, ax = plt.subplots(figsize=(10, 6), dpi=120)
84
85 ax.scatter(
86     t_data,
87     y_data,
88     s=60,
89     color="#27d643",
90     edgecolor="white",
91     linewidth=0.8,
92     alpha=0.9,
93     label="Data points",
94     zorder=3,
95 )
96 ax.plot(
97     t_fit,
98     y_fit,
99     color="#1f77b4",
100    linewidth=2.5,
101    label="Fitted curve",
102    zorder=2,
103 )
104
105 ax.set_xlabel("t", fontsize=12)
106 ax.set_ylabel("y", fontsize=12)
107 ax.set_title("Gauss-Newton Exponential Fit", fontsize=15, weight="bold",
108     ↪ pad=12)
109 ax.legend(frameon=True, fancybox=True, framealpha=0.95, shadow=True)
```

```

108     ax.grid(True, linestyle="--", linewidth=0.6, alpha=0.7)
109
110     ax.spines["top"].set_visible(False)
111     ax.spines["right"].set_visible(False)
112
113     fig.tight_layout()
114     fig.savefig("gauss_newton_exponential_fit.png", dpi=300,
        ↪     bbox_inches="tight")

```



Parameter set	Values
Initial guess $c^{(0)}$	[1.0, 1.0]
Optimized parameters c	[0.96590464, 0.5465158]

Exercise 2 (Linearization by taking logarithms)

Use the same data and model as in Problem 1, but now fit the model through logarithmic linearization.

Part (a)

Starting from

$$y_i = c_1 t_i^{c_2}, \quad i = 1, \dots, N,$$

and using the assumptions $y_i > 0$ and $t_i > 0$, take logarithms of both sides:

$$\log y_i = \log c_1 + c_2 \log t_i.$$

With the substitutions

$$z_i = \log t_i, \quad w_i = \log y_i, \quad \alpha = \log c_1, \quad \beta = c_2,$$

the model becomes

$$w_i = \alpha + \beta z_i, \quad i = 1, \dots, N.$$

Therefore, the fitting problem is to find α and β that minimize

$$\sum_{i=1}^N (w_i - (\alpha + \beta z_i))^2.$$

This is a linear least-squares problem in the unknown vector

$$u = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}.$$

In matrix form,

$$Xu \approx w,$$

where

$$X = \begin{bmatrix} 1 & z_1 \\ 1 & z_2 \\ \vdots & \vdots \\ 1 & z_N \end{bmatrix}, \quad u = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}, \quad w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_N \end{bmatrix}.$$

Thus, the linear least-squares problem is

$$\min_{u \in \mathbb{R}^2} \|Xu - w\|_2^2.$$

Comparing with the generic form, we have

$$x_{i,1} = 1, \quad x_{i,2} = z_i, \quad \theta_1 = \alpha, \quad \theta_2 = \beta, \quad \hat{y}_i = w_i.$$

After solving for u , recover the original parameters by

$$c_1 = e^\alpha, \quad c_2 = \beta.$$

Part (b)

Define

$$E_{\log}(u) = \frac{1}{2} \|Xu - w\|_2^2 = \frac{1}{2} (Xu - w)^T (Xu - w).$$

Then

$$F_{\log}(u) = \nabla E_{\log}(u) = X^T (Xu - w) = X^T Xu - X^T w.$$

Hence the stationarity condition is

$$F_{\log}(u)^T = 0 \iff F_{\log}(u) = 0 \iff X^T (Xu - w) = 0 \iff X^T Xu = X^T w.$$

Assuming the observations are independent, the likelihood is

$$J(\theta) = \prod_{i=1}^N [\alpha(\theta^T x_i)]^{y_i} [1 - \alpha(\theta^T x_i)]^{1-y_i}.$$

Taking logs gives the log-likelihood

$$\ell(\theta) = \log J(\theta) = \sum_{i=1}^N \left[y_i \log \alpha(\theta^T x_i) + (1 - y_i) \log (1 - \alpha(\theta^T x_i)) \right].$$

For a single sample (x_i, y_i) , let

$$\ell^{(i)}(\theta) = y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i), \quad \hat{y}_i = \alpha(\theta^T x_i).$$

Using the chain rule,

$$\frac{\partial \ell^{(i)}(\theta)}{\partial \theta_j} = \frac{\partial \ell^{(i)}}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial (\theta^T x_i)} \cdot \frac{\partial (\theta^T x_i)}{\partial \theta_j}.$$

Now,

$$\frac{\partial \ell^{(i)}}{\partial \hat{y}_i} = \frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{1 - \hat{y}_i}, \quad \frac{\partial \hat{y}_i}{\partial (\theta^T x_i)} = \hat{y}_i(1 - \hat{y}_i), \quad \frac{\partial (\theta^T x_i)}{\partial \theta_j} = x_{ij}.$$

Therefore,

$$\frac{\partial \ell^{(i)}(\theta)}{\partial \theta_j} = \left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{1 - \hat{y}_i} \right) \hat{y}_i(1 - \hat{y}_i)x_{ij} = (y_i - \hat{y}_i)x_{ij}.$$

Equivalently, in vector form,

$$\nabla \ell^{(i)}(\theta) = (y_i - \hat{y}_i)x_i.$$

If instead $J^{(i)}(\theta)$ denotes the *negative* log-likelihood (the cost minimized in logistic regression), then

$$\frac{\partial J^{(i)}(\theta)}{\partial \theta_j} = (\hat{y}_i - y_i)x_{ij}, \quad \nabla J^{(i)}(\theta) = (\hat{y}_i - y_i)x_i.$$

Part (c)

Recover the original parameters from

$$c_1 = e^\alpha, \quad c_2 = \beta.$$

If possible, compare the fitted parameters from Problems 1 and 2 and explain briefly why the two answers need not agree.

Part (d)

If possible, implement the log-linearized approach in your favorite language and compare the resulting fit with the Gauss-Newton fit from Problem 1.

Exercise 3 (Rank-one and low-rank inverse updates)

Part (a)

Let $c, d \in \mathbb{R}^n$ and assume that

$$1 + d^T c \neq 0.$$

Verify directly that

$$(I + cd^T)^{-1} = I - \frac{cd^T}{1 + d^T c}.$$

That is, multiply $(I + cd^T)$ by the proposed inverse and simplify.

$$\begin{aligned} (I + cd^T) \left(I - \frac{cd^T}{1 + d^T c} \right) &= I + cd^T - \frac{cd^T}{1 + d^T c} - \frac{cd^T cd^T}{1 + d^T c} \\ &= I + cd^T - \frac{cd^T}{1 + d^T c} - \frac{(d^T c)cd^T}{1 + d^T c} \\ &= I + cd^T - \frac{(1 + d^T c)cd^T}{1 + d^T c} \\ &= I \end{aligned}$$

Part (b)

Let $A \in \mathbb{R}^{n \times n}$ be nonsingular, and let $C, D \in \mathbb{R}^{n \times k}$. Assume that

$$I + D^T A^{-1} C$$

is nonsingular. Verify the Sherman-Morrison-Woodbury formula

$$(A + CD^T)^{-1} = A^{-1} - A^{-1}C(I + D^T A^{-1}C)^{-1}D^T A^{-1}.$$

Again, verify this by multiplying $(A + CD^T)$ by the inverse and simplifying.

$$\begin{aligned} \text{Let } M &= I + D^T A^{-1} C. \\ (A + CD^T) (A^{-1} - A^{-1}CM^{-1}D^T A^{-1}) &= AA^{-1} - AA^{-1}CM^{-1}D^T A^{-1} + CD^T A^{-1} - CD^T A^{-1}CM^{-1}D^T A^{-1} \\ &= I - CM^{-1}D^T A^{-1} + CD^T A^{-1} - CD^T A^{-1}CM^{-1}D^T A^{-1} \\ &= I + CD^T A^{-1} - C(M^{-1} + D^T A^{-1}CM^{-1})D^T A^{-1} \\ &= I + CD^T A^{-1} - C(I + D^T A^{-1}C)M^{-1}D^T A^{-1} \\ &= I + CD^T A^{-1} - CD^T A^{-1} \\ &= I. \end{aligned}$$

Hence

$$(A + CD^T)^{-1} = A^{-1} - A^{-1}C(I + D^T A^{-1}C)^{-1}D^T A^{-1}.$$

Part (c)

Explain briefly why part (a) is a special case of part (b).

$$\begin{aligned} \text{Let: } A = I, \quad C = c, \quad D = d, \quad k = 1 \quad \text{st} \quad CD^T = cd^T, \quad I + D^T A^{-1} C = 1 + d^T c, \\ \implies (I + cd^T)^{-1} = I - \frac{cd^T}{1 + d^T c}. \end{aligned}$$

Exercise 4

Let $A \in \mathbb{R}^{n \times n}$ be a symmetric positive definite matrix and $b \in \mathbb{R}^n$. Consider the quadratic functional

$$J(x) = \frac{1}{2}x^T A x - x^T b, \quad x \in \mathbb{R}^n.$$

It is known that minimizing J is equivalent to solving the linear system

$$Ax = b.$$

Verify this by computing the partial derivative of J with respect to x_k , and showing that

$$\frac{\partial J}{\partial x_k}(x) = 0$$

is exactly the k -th equation of the system $Ax = b$, for each $k = 1, \dots, n$.

$$J(x) = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n a_{ij} x_i x_j - \sum_{i=1}^n b_i x_i,$$

and use that A is symmetric.

$$\begin{aligned} \frac{\partial J}{\partial x_k}(x) &= \frac{\partial}{\partial x_k} \left(\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n a_{ij} x_i x_j - \sum_{i=1}^n b_i x_i \right) \\ &= \frac{1}{2} \sum_{j=1}^n a_{kj} x_j + \frac{1}{2} \sum_{i=1}^n a_{ik} x_i - b_k. \end{aligned}$$

Since A is symmetric, $a_{ik} = a_{ki}$. Therefore,

$$\frac{1}{2} \sum_{i=1}^n a_{ik} x_i = \frac{1}{2} \sum_{i=1}^n a_{ki} x_i.$$

Relabeling the index i as j , we obtain

$$\frac{\partial J}{\partial x_k}(x) = \frac{1}{2} \sum_{j=1}^n a_{kj} x_j + \frac{1}{2} \sum_{j=1}^n a_{kj} x_j - b_k = \sum_{j=1}^n a_{kj} x_j - b_k.$$

But

$$\sum_{j=1}^n a_{kj} x_j = (Ax)_k,$$

so

$$\frac{\partial J}{\partial x_k}(x) = (Ax)_k - b_k.$$

Hence,

$$\frac{\partial J}{\partial x_k}(x) = 0 \iff (Ax)_k = b_k \iff \sum_{j=1}^n a_{kj}x_j = b_k,$$

which is exactly the k -th equation of the linear system $Ax = b$. Since this holds for every $k = 1, \dots, n$, minimizing J is equivalent to solving $Ax = b$.